

Sorting in Python



Algoritma dan Pemrograman II

Team Teaching:
Lia Farhatuaini, M.Kom
Sokid, M.Kom

SORTING

- Setelah mempelajari bagaimana mengakses elemen pada list secara efisien, maka selanjutnya kita akan belajar bagaimana mengurutkan elemen-elemen tersebut.
- Telah banyak algoritma yang sudah di kembangkan untuk melakukan pengurutan (sorting)
- Kita akan belajar algoritma sorting yang paling sederhana yaitu Bubble Sort, Algoritma ini sangat mudah digunakan, tetapi tidak terlalu efisien untuk mengurutkan list yang ukurannya besar.
- Untuk materi algoritma sorting lain lebih detail akan dibahas pada MK Struktur Data

THE BUBBLE SORT

- Suatu list dapat diurutkan dengan 2 cara: **increasing** dan **decreasing**
- misalkan kita akan mengurutkan secara **increasing** sebuah list: [8, 10, 6, 2, 4]
- [8, 10, 6, 2, 4] -> [8, 10, 6, 2, 4]
- [8, 10, 6, 2, 4] -> [8, 6, 10, 2, 4]
- [8, 6, 10, 2, 4] -> [8, 6, 2, 10, 4]
- [8, 6, 2, 10, 4] -> [8, 6, 2, 4, 10] - elemen paling tinggi sudah pada tempatnya
- [8, 6, 2, 4, 10] -> [6, 8, 2, 4, 10]
- [6, 8, 2, 4, 10] -> [6, 2, 8, 4, 10]
- [6, 2, 8, 4, 10] -> [6, 2, 4, 8, 10] - 2 elemen sudah pada tempatnya
- [6, 2, 4, 8, 10] -> [2, 6, 4, 8, 10]
- [2, 6, 4, 8, 10] -> [2, 4, 6, 8, 10] - 3 elemen sudah pada tempatnya
- [2, 4, 6, 8, 10] -> [2, 4, 6, 8, 10] - semua elemen sudah pada tempatnya/ terurut
- Jadi ada berapa langkah untuk mengurutkan 5 elemen list?

SORTING A LIST

- untuk melakukan sorting, kita membutuhkan variabel baru untuk mengindikasikan apakah terjadi proses penukaran atau tidak. kita namai variabel tersebut dengan **swapped**, kita berikan nilai False jika tidak terjadi penukaran dan nilai True jika terjadi penukaran.

```
1  my_list = [8, 10, 6, 2, 4] # list untuk diurutkan
2  swapped = True # Inisiasi awal, untuk memasuki loop
3
4  while swapped:
5      swapped = False # Untuk mengindikasi bahwa tidak ada proses penukaran elemen
6      for i in range(len(my_list) - 1): # Melakukan perulangan sepanjang banyaknya elemen dikurang 1
7          if my_list[i] > my_list[i + 1]: # Membandingkan elemen saat ini dengan elemen di depannya
8              swapped = True # Untuk mengindikasi bahwa akan ada proses penukaran
9              my_list[i], my_list[i + 1] = my_list[i + 1], my_list[i] # Proses penukaran
10
11 print(my_list) # Mencetak isi list
```

VERSI INTERAKTIF BUBBLE SORT

- Jika dibuat lebih interaktif terhadap user, algoritma bubble sort secara komplit dapat dilihat pada source code berikut

```
1  my_list = []
2  swapped = True
3  num = int(input("Masukkan panjang elemen list yang akan diurutkan: "))
4
5  for i in range(num):
6      val = float(input("Masukkan elemen list: "))
7      my_list.append(val)
8
9  while swapped:
10     swapped = False
11     for i in range(len(my_list) - 1):
12         if my_list[i] > my_list[i + 1]:
13             swapped = True
14             my_list[i], my_list[i + 1] = my_list[i + 1], my_list[i]
15
16  print("\nSorted:")
17  print(my_list)
```


READY-TO-USE TOOLS

- Python memiliki method yang bisa langsung digunakan untuk melakukan sorting, sehingga kita tidak perlu lagi membuat sorting sendiri.
- penggunaan method untuk sorting dapat dilihat pada source code berikut:

```
1  my_list = [8, 10, 6, 2, 4]
2  my_list.sort()
3  print(my_list)
```

- Python juga memiliki method lain yang bisa langsung digunakan untuk membalik urutan dari list yang kita miliki

```
1  lst = [5, 3, 1, 2, 4]
2  lst.reverse()
3  print(lst)
```

Operations on List in Python



Algoritma dan Pemrograman II

Team Teaching:
Lia Farhatuaini, M.Kom
Sokid, M.Kom

THE INNER LIFE OF LISTS-1

- List (dan banyak entitas lain yg kompleks pada Python) disimpan dengan cara yang berbeda dari pada variabel (scalar) yg biasanya.

- perhatikan kode berikut:

```
1 list_1 = [1]
2 list_2 = list_1
3 list_1[0] = 2
4 print(list_2)
```

- Baris 1: membuat list dengan nama list_1 dengan satu elemen [1]
- Baris 2: membuat list baru list_2 dan mengisinya dengan list_1
- Baris 3: mengubah elemen dari index 0 dari list_1 dengan 2
- Baris 4: Mencetak isi list_2

- Kira-kira apa outputnya?

THE INNER LIFE OF LISTS-2

- Mengejutkan karena output dari kode pada slide sebelumnya adalah [2] bukan [1]. kenapa? karena:
- Nama dari suatu variabel merupakan nama dari **conten/isi** dari variabel tsb
- Nama dari suatu list merupakan **lokasi memori** dimana list tersebut disimpan
- Sehingga kode `list_2 = list_1`, bukan menyalin isi dari `list_1` ke `list_2`. tetapi berarti lokasi memori dari `list_2` sama dengan lokasi memori dari `list_1`.
- Sehingga memodifikasi isi dari `list_1` akan berdampak pada `list_2`, begitu pula sebaliknya.

SLICE

- Sebuah slice merupakan bagian dari sintaksis python yang memungkinkan kita membuat salinan baru dari sebuah list atau bagian dari sebuah list

```
1 list_1 = [1]
2 list_2 = list_1[:]
3 list_1[0] = 2
4 print(list_2)
```

- bagian kode `[:]` dapat membuat suatu list baru dengan mengcopy semua elemen dari list sumber
- `my_list[awal:akhir]`
- sebuah slice dari bentuk di atas dapat membuat list target yang baru yang elemennya diambil dari list sumber dimulai dari elemen **awal** sampai **akhir-1**

```
1 my_list = [10, 8, 6, 4, 2]
2 new_list = my_list[1:3]
3 print(new_list)
```

SLICE DENGAN INDEX NEGATIF

- Sekali lagi, slice `my_list[awal:akhir]` berarti:
- elemen awal= dimasukkan pada list baru
- elemen akhir = tidak dimasukkan pada list baru

```
1  my_list = [10, 8, 6, 4, 2]
2  new_list = my_list[1:-1]
3  print(new_list)
```

```
1  my_list = [10, 8, 6, 4, 2]
2  new_list = my_list[-1:1]
3  print(new_list)
```


SLICE DENGAN INDEX NEGATIF

- kita bisa saja menghilangkan awal dari slice, `my_list[:akhir]`. ini diasumsikan bahwa kita akan membuat slice dimulai dari index 0 sampai akhir
- `my_list[:akhir]` sama dengan `my_list[0:akhir]`

```
1 my_list = [10, 8, 6, 4, 2]
2 new_list = my_list[:3]
3 print(new_list)
```

- Kita juga bisa menghilangkan akhir dari slice, `my_list[awal:]`. ini diasumsikan bahwa kita akan membuat slice dari awal sampai panjang dari list/ `len(list)`
- `my_list[awal:]` sama dengan `my_list[awal:len(my_list)]`

```
1 my_list = [10, 8, 6, 4, 2]
2 new_list = my_list[3:]
3 print(new_list)
```

INSTRUKSI DEL

- Instruksi del dapat digunakan juga untuk menghapus suatu slice

```
1 my_list = [10, 8, 6, 4, 2]
2 del my_list[1:3]
3 print(my_list)
```

- dalam kasus ini, slice tidak membuat suatu list baru tetapi menghapus elemen dari list sumber
- kita juga bisa menghapus semua elemen dari list dalam satu langkah

```
1 my_list = [10, 8, 6, 4, 2]
2 del my_list[:]
3 print(my_list)
```

- tidak menyertakan slice dari kode del berarti kita menghapus list tersebut bukan hanya isi dari listnya

```
1 my_list = [10, 8, 6, 4, 2]
2 del my_list
3 print(my_list)
```

OPERATOR IN DAN NOT IN

- operator in dan not in akan mengecek apakah elemen yang diberikan ada pada list.
- operator in akan mengembalikan nilai True jika ada
- operator not in akan mengembalikan nilai True jika tidak ada

```
1  my_list = [0, 3, 12, 8, 2]
2
3  print(5 in my_list)
4  print(5 not in my_list)
5  print(12 in my_list)
```


SIMPLE PROGRAM DARI LIST-1

- Jelaskan kode di bawah

```
1  my_list = [17, 3, 11, 5, 1, 9, 7, 15, 13]
2  largest = my_list[0]
3
4  for i in range(1, len(my_list)):
5      |   if my_list[i] > largest:
6      |       |   largest = my_list[i]
7
8  print(largest)
```

SIMPLE PROGRAM DARI LIST-2

- Jelaskan kode di bawah

```
1  my_list = [17, 3, 11, 5, 1, 9, 7, 15, 13]
2  largest = my_list[0]
3
4  for i in my_list:
5      |   if i > largest:
6      |       |   largest = i
7
8  print(largest)
```

SIMPLE PROGRAM DARI LIST-3

- Jelaskan kode di bawah

```
1  my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2  to_find = 5
3  found = False
4
5  for i in range(len(my_list)):
6      found = my_list[i] == to_find
7      if found:
8          break
9
10 if found:
11     print("Elemen ditemukan pada index ke-", i)
12 else:
13     print("Tidak ada di dalam list")
14
```

KUIS 21

- Pada suatu hari kamu menebak 6 angka pada lotre yaitu: 3, 7, 11, 42, 34, 49
- angka yang sudah keluar diundi adalah 5, 9, 11, 42, 3, 49
- berapa kali anda menebak dengan benar?
- buatlah kode pythonnya

KUIS 22

- Jika terdapat sebuah list dengan isi: [1, 2, 4, 4, 1, 4, 2, 6, 2, 9]
- buatlah program python untuk menghilangkan angka yang berulang yang ada pada list tersebut, sehingga setiap angka pada list merupakan elemen yang uniq/tidak muncul lebih dari satu kali
- hint: gunakan list baru sebagai temporary work area